



Progressive Education Society's
MODERN COLLEGE OF ENGINEERING, Pune -05.
(An Autonomous Institute Affiliated to Savitribai Phule Pune University)
MCA Department

PRACTICAL SUBMISSION RECORD- A.Y. 2025-26

Class: SYMCA	Div.: B	Course Code: MCA01604	Batch: S2
Semester: III		Course Name: Data Science Laboratory	
Name: Ajit Bajarang Mali		Roll No: 52140	
CO No: CO605.1		Assignment No: 3	

Title: Use the Naive Bayes classifier to perform sentiment analysis on a dataset of movie reviews, classifying them as either positive or negative based on the review text.

Code:-

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

1. Create a sample dataset of movie reviews

```
data = {
    'review': [
        'This movie was fantastic! The acting was superb.',
        'A truly terrible film. I would not recommend it.',
        'An amazing and inspiring story. A must-see.',
        'Utterly boring and a complete waste of time.',
        'The plot was predictable, but the visuals were stunning.',
        'I loved it! So funny and heartwarming.',
        'One of the worst movies I have ever seen.',
        'A masterpiece of cinema. Absolutely brilliant.',
        'The script was weak and the characters were underdeveloped.',
        'An enjoyable ride from start to finish.'
    ],
    'sentiment': [
        'positive', 'negative', 'positive', 'negative', 'positive',
        'positive', 'negative', 'positive', 'negative', 'positive'
    ]
}
df = pd.DataFrame(data)
```

2. Prepare the data

```
# Convert text data into numerical vectors using TF-IDF
vectorizer = TfidfVectorizer(stop_words='english')
X = vectorizer.fit_transform(df['review'])
y = df['sentiment']
```

Split data into training and testing sets

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```



```
# 3. Train the Naive Bayes Classifier
# We use the Multinomial Naive Bayes, which is suitable for text classification
model = MultinomialNB()
model.fit(X_train, y_train)

# 4. Evaluate the model's performance
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

# Generate a confusion matrix
cm = confusion_matrix(y_test, y_pred, labels=['positive', 'negative'])
class_report = classification_report(y_test, y_pred)

# 5. Display the results
print("--- Model Evaluation ---")
print(f"Accuracy: {accuracy:.2f}")
print("\nClassification Report:\n", class_report)
print("\n--- Confusion Matrix ---")

# Plotting the confusion matrix
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['positive', 'negative'],
            yticklabels=['positive', 'negative'])
plt.xlabel('Predicted Sentiment')
plt.ylabel('Actual Sentiment')
plt.title('Confusion Matrix')
plt.show()
```

Output:-

```
--- Model Evaluation ---
Accuracy: 0.33
```

```
Classification Report:
              precision    recall  f1-score   s

   negative       0.00      0.00      0.00
   positive       0.33      1.00      0.50

 accuracy              0.33
 macro avg       0.17      0.50      0.25
 weighted avg     0.11      0.33      0.17
```

